CSC 4814 & 6814 Efficient AI

Lecture 03

Dr. Haoxin Wang

Assistant Professor, GSU

https://www.amai-gsu.us/

haoxinwang@gsu.edu

This course draws inspiration and selected materials from Prof. Song Han's MIT course on TinyML and Efficient Deep Learning Computing. Credit is gratefully acknowledged.

Deep Learning is Everywhere

- **But they are computationally costly. How to make them light-weighted and fast?**







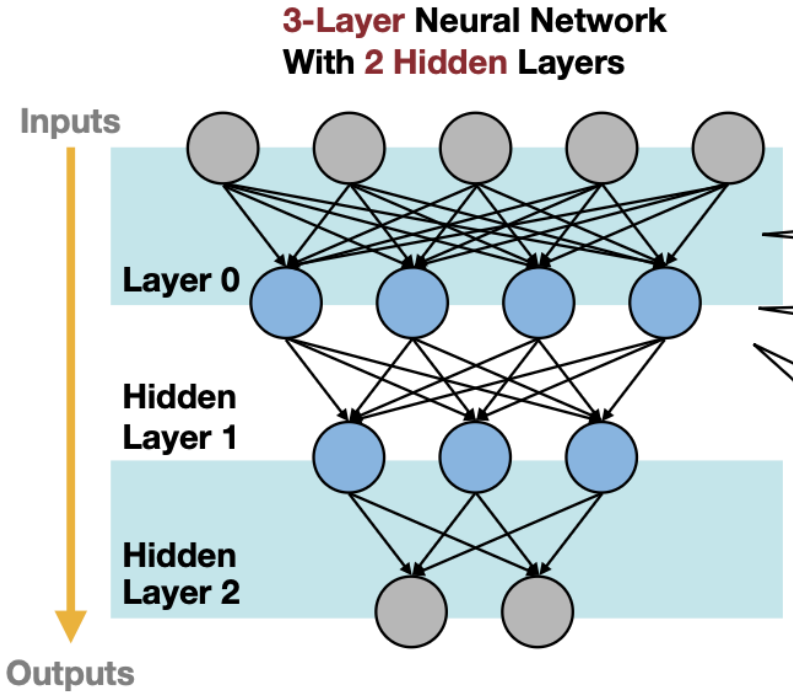
This course draws inspiration and selected materials from Prof. Song Han's MIT course on TinyML and Efficient Deep Learning Computing. Credit is gratefully acknowledged.

Plan for Lectures 2-3

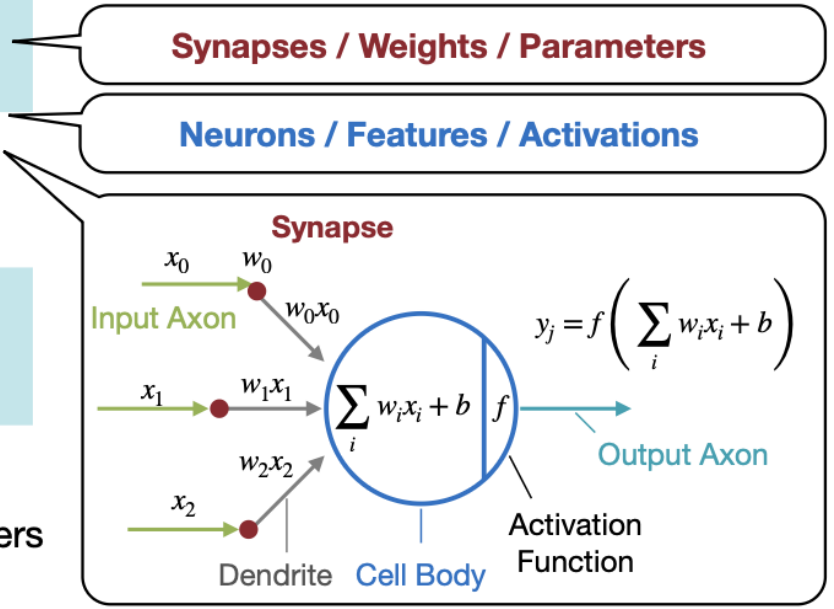
1. Review the **terminology of neural networks**
   - Neuron, Synapses, Activation, Feature, Weight, Parameter, *etc*
2. Review **popular building blocks** in a neural network
   - Fully-Connected, Convolution, Grouped Convolution, Depthwise Convolution
   - Pooling, Normalization, Transformer
3. Introduce popular efficiency metrics for neural networks
   - #Parameters, Model Size, Peak #Activations, MAC, FLOP, FLOPS, Latency, Throughput



Recap: Deep Neural Network



The dimensionality of these *hidden* layers determines the **width** of the model.



Popular Neural Network Layers



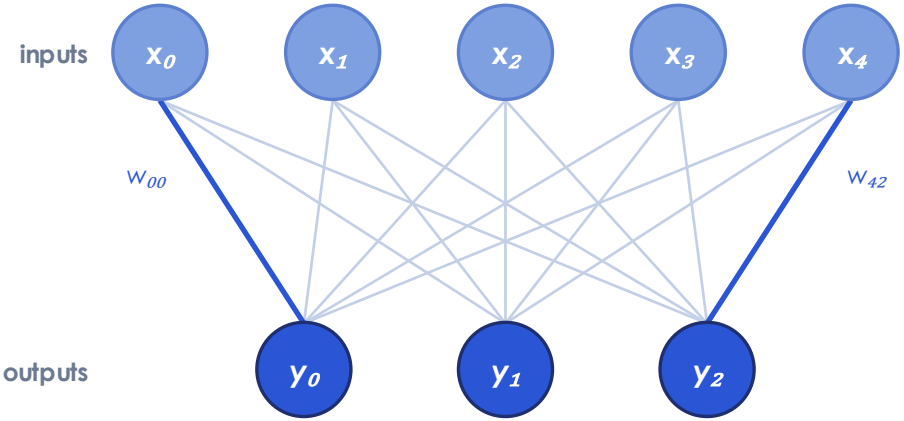
Recap: Fully-Connected Layer

Every output neuron connects to every input neuron

The layer that starts it all: no connection is skipped. Once you see it as a single matrix multiply, the whole thing collapses into one line of math.

$$Y = XW^T + b$$

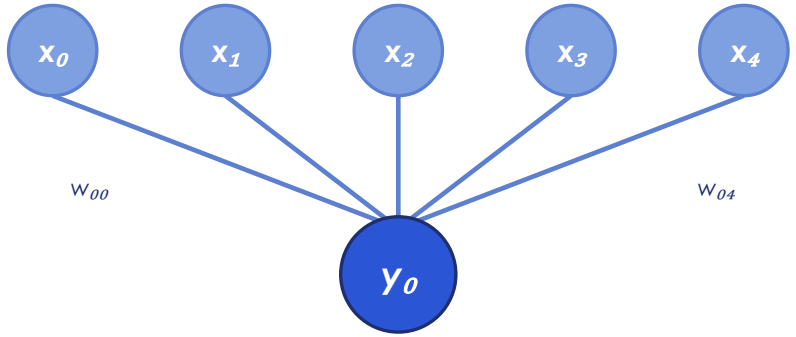
one matrix multiply, plus a bias



Recap: One Neuron = a Weighted Sum

Give each input a weight, add them up, add a bias

Intuition. Think of x as 5 features of a person (height, weight, age…). To score "fits a basketball team," weight each feature by how much it matters, sum them, then shift by a bias baseline.



$$y_0 = w_{00}\,x_0 + w_{01}\,x_1 + w_{02}\,x_2 + w_{03}\,x_3 + w_{04}\,x_4 + b_0$$

In general, for every output i: $y_i = \Sigma_j\, w_{ij}\, x_j + b_i$

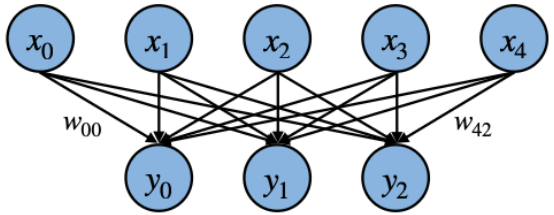
the Σ just compresses the line above

Recap: Fully-Connected Layer (Linear layer)

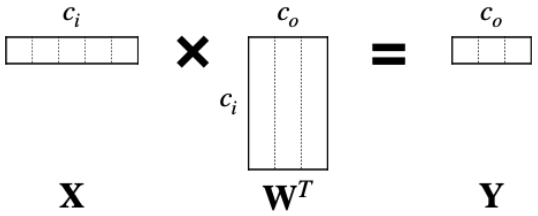
The output neuron is connected to all input neurons.

- **Shape of Tensors:**
  - Input Features $\mathbf{X}$: $(1, c_i)$
  - Output Features $\mathbf{Y}$: $(1, c_o)$
  - Weights $\mathbf{W}$: (c_o, c_i)
  - Bias $\mathbf{b}$: $(c_o,)$

Notations	
c_i	Input Channels
c_o	Output Channels



$$y_i = \sum_j w_{ij} x_j + b_i$$

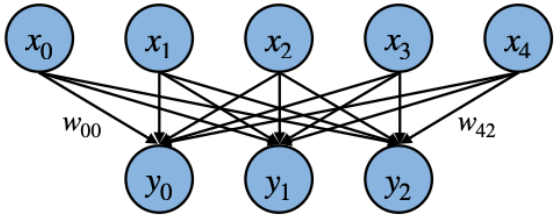


Recap: Fully-Connected Layer (Linear layer)

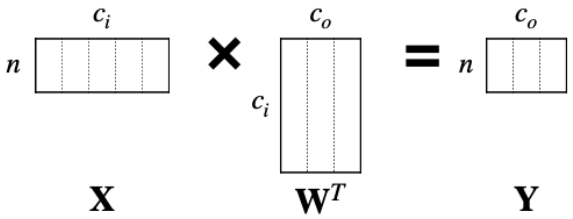
The output neuron is connected to all input neurons.

- **Shape of Tensors:**
  - Input Features $\mathbf{X}$: (n, c_i)
  - Output Features $\mathbf{Y}$: (n, c_o)
  - Weights $\mathbf{W}$: (c_o, c_i)
  - Bias $\mathbf{b}$: $(c_o,)$

Notations	
n	Batch Size
c_i	Input Channels
c_o	Output Channels



$$y_i = \sum_j w_{ij} x_j + b_i$$

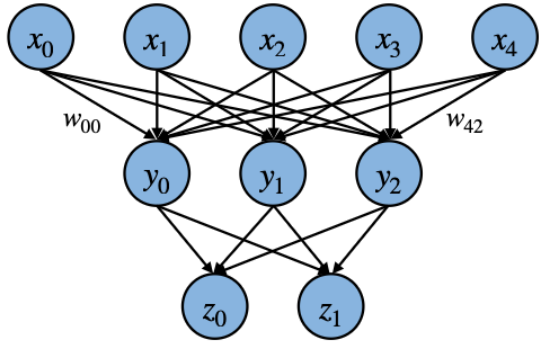


Recap: Fully-Connected Layer (Linear layer)

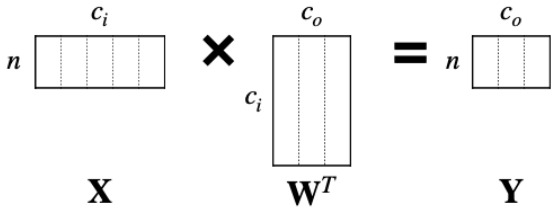
The output neuron is connected to all input neurons.

- **Shape of Tensors:**
  - Input Features $\mathbf{X}$: (n, c_i)
  - Output Features $\mathbf{Y}$: (n, c_o)
  - Weights $\mathbf{W}$: (c_o, c_i)
  - Bias $\mathbf{b}$: $(c_o,)$

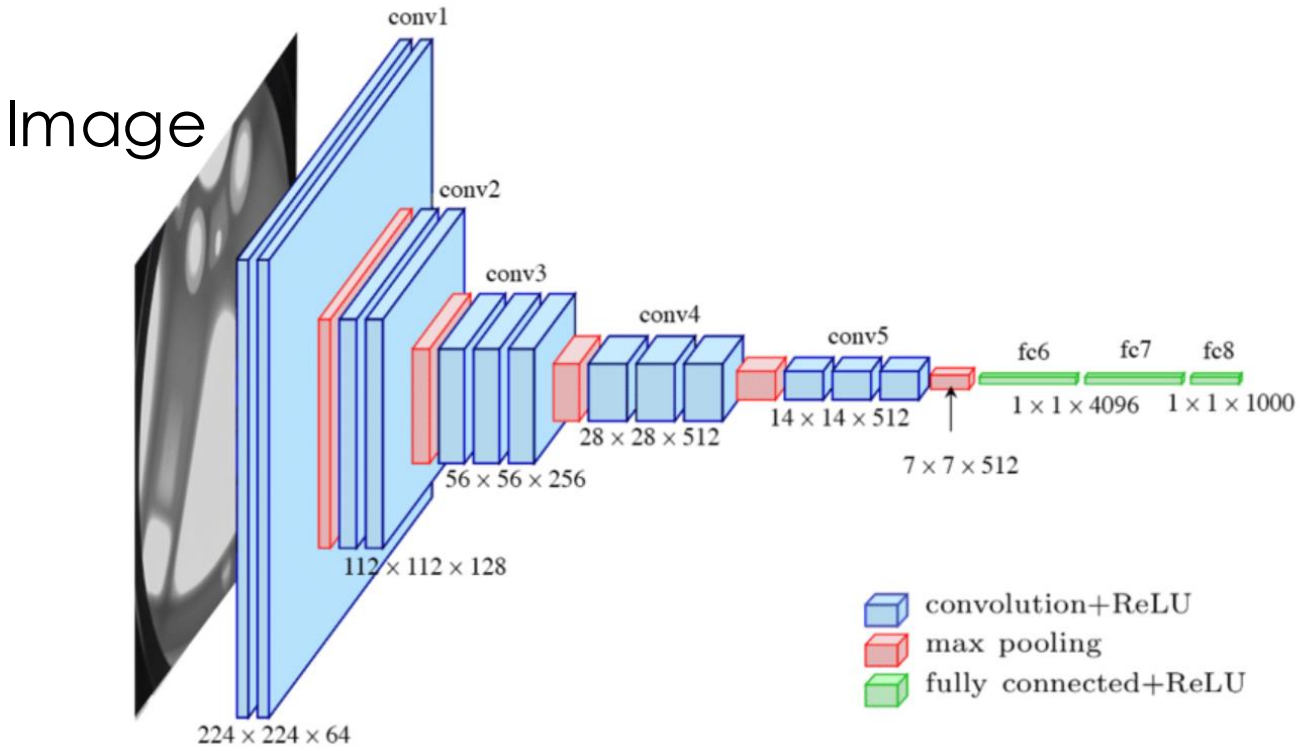
Notations	
n	Batch Size
c_i	Input Channels
c_o	Output Channels



Multilayer Perceptron (MLP)

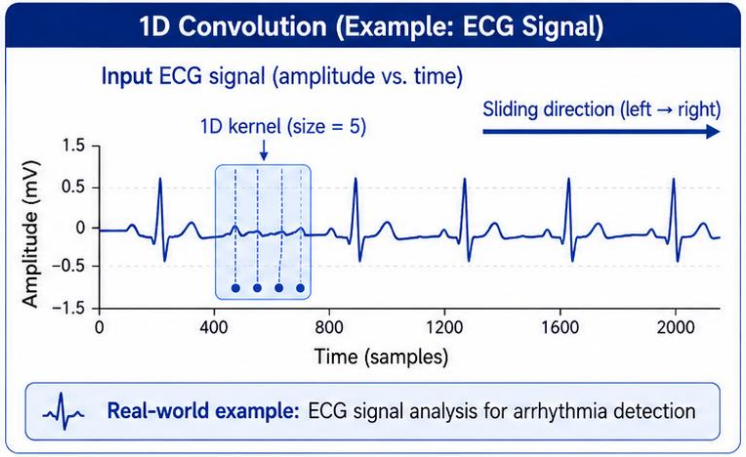


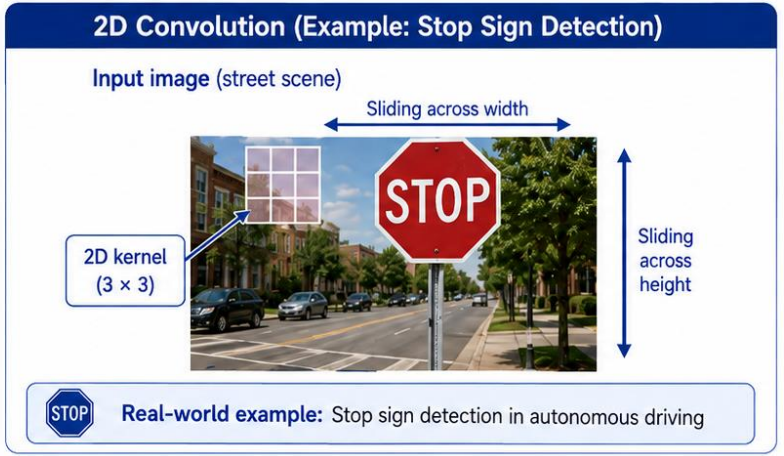
Model Architecture Example



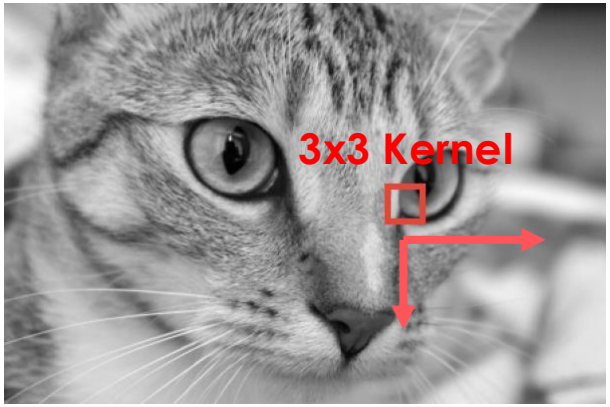
Convolution Layer

- 1D Conv vs. 2D Conv
  - **1D convolution:** the kernel slides along **one dimension**, such as time or sequence length.
  - **2D convolution:** the kernel slides along **two dimensions**, such as the height and width of an image.



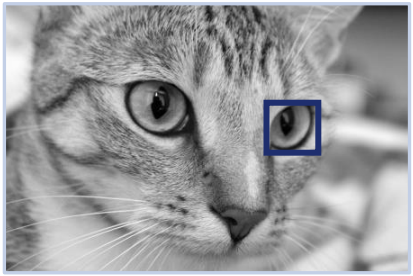


Typical input	Sequence / time series	Images / spatial data
Kernel motion	Moves along time (one dimension)	Moves across height and width (two dimensions)
What it learns	Learns temporal patterns (e.g., spikes, rhythms)	Learns spatial patterns (e.g., edges, textures, shapes)


3x3 Kernel

The Kernel Decides What You See

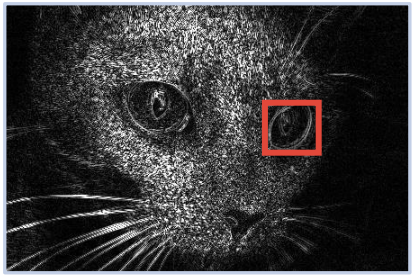
Same photo, four different 3×3 kernels → four different features



Original

0 0 0
0 1 0
0 0 0

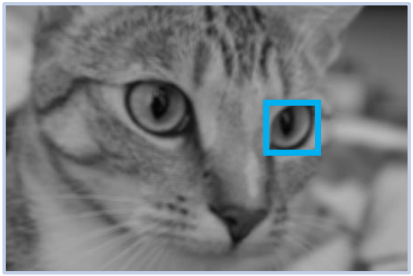
3×3 kernel



Edge detect

-1 -1 -1
-1 8 -1
-1 -1 -1

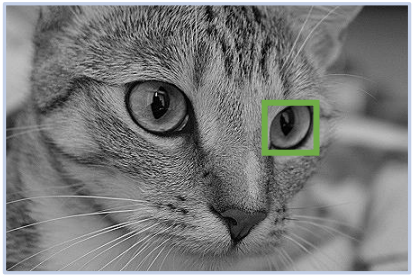
3×3 kernel



Blur

1 1 1
1 1 1
1 1 1

×1/9



Sharpen

0 -1 0
-1 5 -1
0 -1 0

3×3 kernel

- A conv layer learns many filters; each one outputs its own channel;
- Convolution means sliding a small filter over the input and computing a weighted sum at each location.

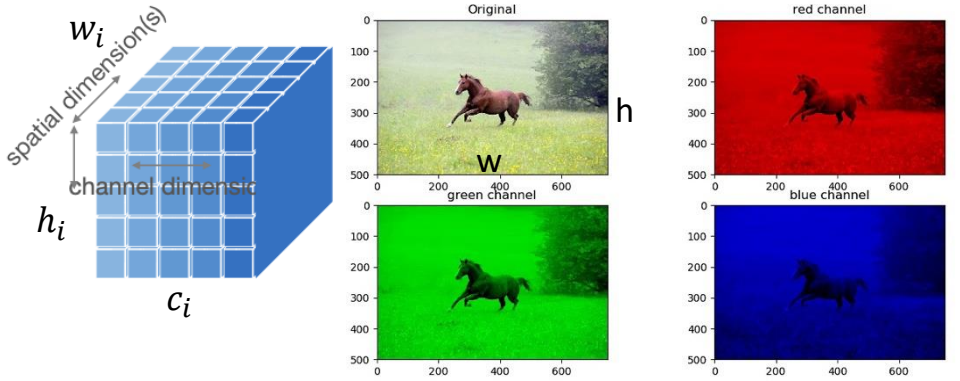
2D Convolution Layer

The output neuron is connected to input neurons in the receptive field.

- **Shape of Tensors:**

	1D Conv	2D Conv
Input Features $\mathbf{X}$: ~~(n, c_i)~~	(n, c_i, w_i)	(n, c_i, h_i, w_i)
Output Features $\mathbf{Y}$: ~~(n, c_o)~~	(n, c_o, w_o)	(n, c_o, h_o, w_o)
Weights $\mathbf{W}$: ~~(c_o, c_i)~~	(c_o, c_i, k_w)	(c_o, c_i, k_h, k_w)
Bias $\mathbf{b}$: $(c_o,)$		

Notations	
n	Batch Size
c_i	Input Channels
c_o	Output Channels
h_i, h_o	Input/Output Height
w_i, w_o	Input/Output Width



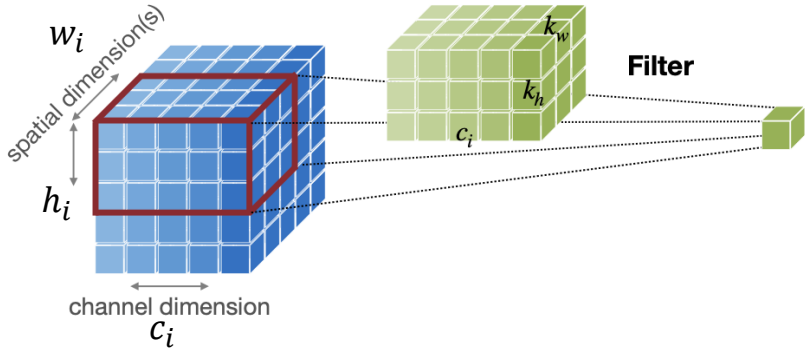
2D Convolution Layer

The output neuron is connected to input neurons in the receptive field.

- **Shape of Tensors:**

	1D Conv	2D Conv
• Input Features $\mathbf{X}$: ~~(n, c_i)~~	(n, c_i, w_i)	(n, c_i, h_i, w_i)
• Output Features $\mathbf{Y}$: ~~(n, c_o)~~	(n, c_o, w_o)	(n, c_o, h_o, w_o)
• Weights $\mathbf{W}$: ~~(c_o, c_i)~~	(c_o, c_i, k_w)	(c_o, c_i, k_h, k_w)
• Bias $\mathbf{b}$: $(c_o,)$		

Notations	
n	Batch Size
c_i	Input Channels
c_o	Output Channels
h_i, h_o	Input/Output Height
w_i, w_o	Input/Output Width
k_h	Kernel Height
k_w	Kernel Width



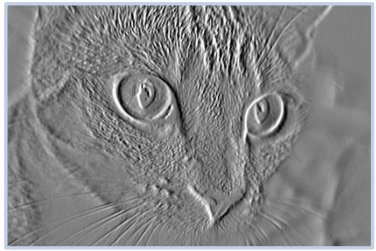
One Filter, One Feature Map

A conv layer learns many filters; each one outputs its own channel

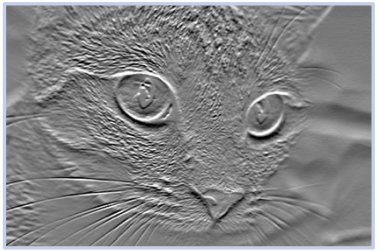


input (3 input channels)

N learned Kernel filters



Output channel 1



Output channel 2



Output channel 3



Output channel 4

= N (number of learned Kernel filters)

Weight tensor shape: $W : (c_o, c_i, k, k)$, one k×k kernel per (output channel × input channel) pair.

= 3 (e.g., RGB image; number of channels in input data)

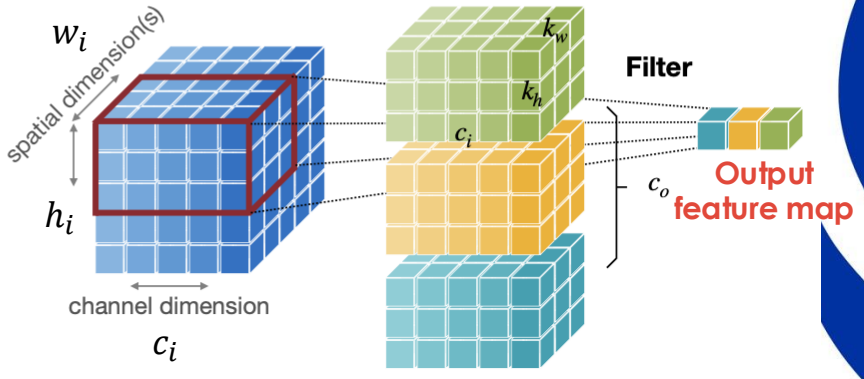
2D Convolution Layer

The output neuron is connected to input neurons in the receptive field.

- **Shape of Tensors:**

	1D Conv	2D Conv
• Input Features $\mathbf{X}$: ~~(n, c_i)~~	(n, c_i, w_i)	(n, c_i, h_i, w_i)
• Output Features $\mathbf{Y}$: ~~(n, c_o)~~	(n, c_o, w_o)	(n, c_o, h_o, w_o)
• Weights $\mathbf{W}$: ~~(c_o, c_i)~~	(c_o, c_i, k_w)	(c_o, c_i, k_h, k_w)
• Bias $\mathbf{b}$: $(c_o,)$		

Notations	
n	Batch Size
c_i	Input Channels
c_o	Output Channels
h_i, h_o	Input/Output Height
w_i, w_o	Input/Output Width
k_h	Kernel Height
k_w	Kernel Width

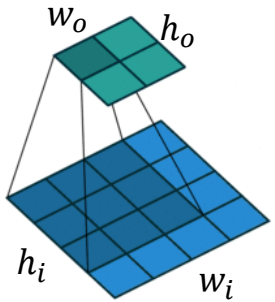


2D Convolution Layer

The output neuron is connected to input neurons in the receptive field.

- **Shape of Tensors:**

	1D Conv	2D Conv
• Input Features $\mathbf{X}$: ~~(n, c_i)~~	(n, c_i, w_i)	(n, c_i, h_i, w_i)
• Output Features $\mathbf{Y}$: ~~(n, c_o)~~	(n, c_o, w_o)	(n, c_o, h_o, w_o)
• Weights $\mathbf{W}$: ~~(c_o, c_i)~~	(c_o, c_i, k_w)	(c_o, c_i, k_h, k_w)
• Bias $\mathbf{b}$: $(c_o,)$		



Notations	
n	Batch Size
c_i	Input Channels
c_o	Output Channels
h_i, h_o	Input/Output Height
w_i, w_o	Input/Output Width
k_h	Kernel Height
k_w	Kernel Width

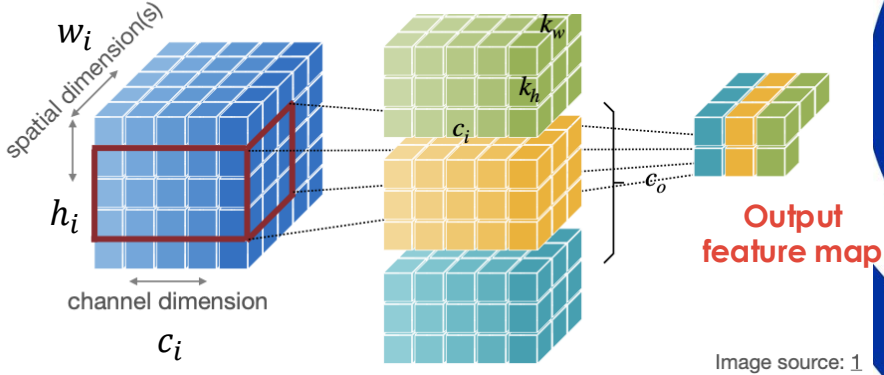


Image source: 1

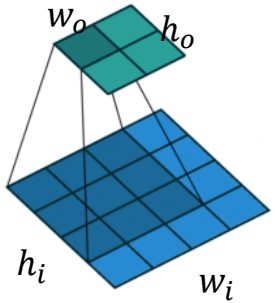
2D Convolution Layer

The output neuron is connected to input neurons in the receptive field.

- **Shape of Tensors:**

	1D Conv	2D Conv
• Input Features $\mathbf{X}$: ~~(n, c_i)~~	(n, c_i, w_i)	(n, c_i, h_i, w_i)
• Output Features $\mathbf{Y}$: ~~(n, c_o)~~	(n, c_o, w_o)	(n, c_o, h_o, w_o)
• Weights $\mathbf{W}$: ~~(c_o, c_i)~~	(c_o, c_i, k_w)	(c_o, c_i, k_h, k_w)
• Bias $\mathbf{b}$: $(c_o,)$		

Notations	
n	Batch Size
c_i	Input Channels
c_o	Output Channels
h_i, h_o	Input/Output Height
w_i, w_o	Input/Output Width
k_h	Kernel Height
k_w	Kernel Width



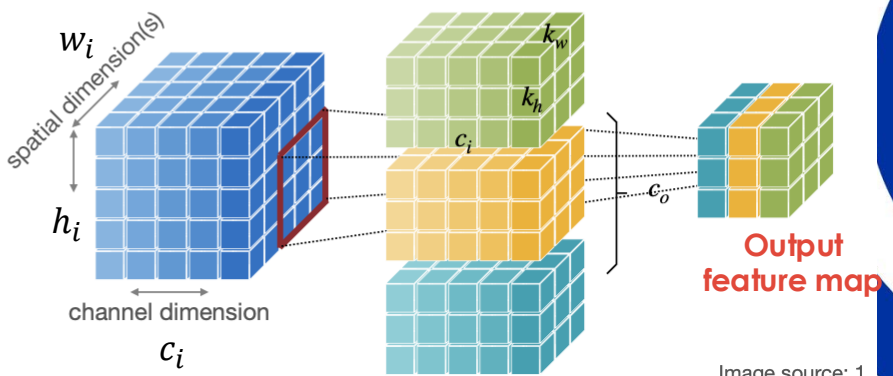


Image source: 1

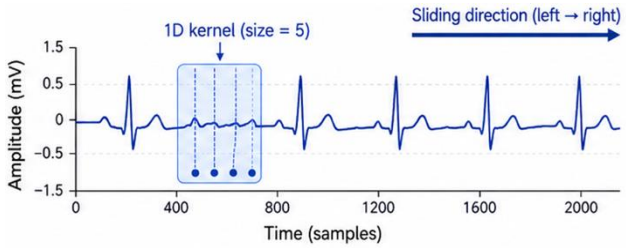
1D Convolution Layer

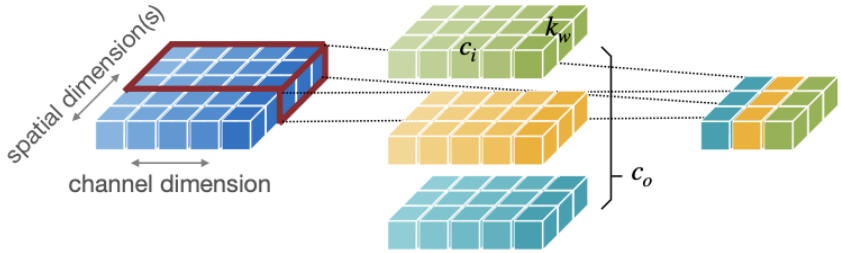
The output neuron is connected to input neurons in the receptive field.

- **Shape of Tensors:**

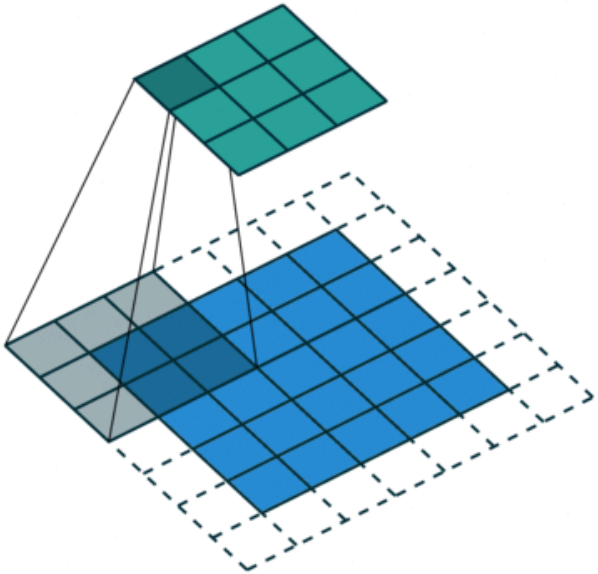
	1D Conv
Input Features $\mathbf{X}$: ~~(n, c_i)~~	(n, c_i, w_i)
Output Features $\mathbf{Y}$: ~~(n, c_o)~~	(n, c_o, w_o)
Weights $\mathbf{W}$: ~~(c_o, c_i)~~	(c_o, c_i, k_w)
Bias $\mathbf{b}$: $(c_o,)$	

Notations	
n	Batch Size
c_i	Input Channels
c_o	Output Channels
w_i, w_o	Input/Output Width
k_w	Kernel Width

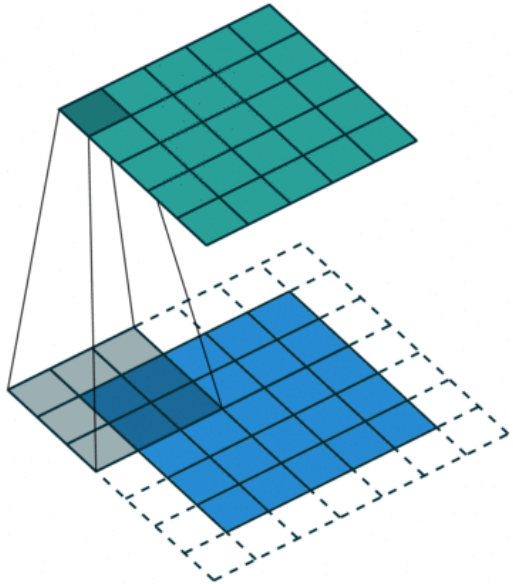




Strided Convolution Layer (Optional)



Stride = 2
Sliding over 9 positions



Stride = 1
Sliding over all 25 possible positions

Grouped Convolution Layer (Optional)

A group of narrower convolutions

- **Shape of Tensors:**
  - Input Features $\mathbf{X}$: (n, c_i, h_i, w_i)
  - Output Features $\mathbf{Y}$: (n, c_o, h_o, w_o)
  - Weights $\mathbf{W}$: ~~(c_o, c_i, k_h, k_w)~~ $(g \cdot c_o/g, c_i/g, k_h, k_w)$
  - Bias $\mathbf{b}$: $(c_o,)$

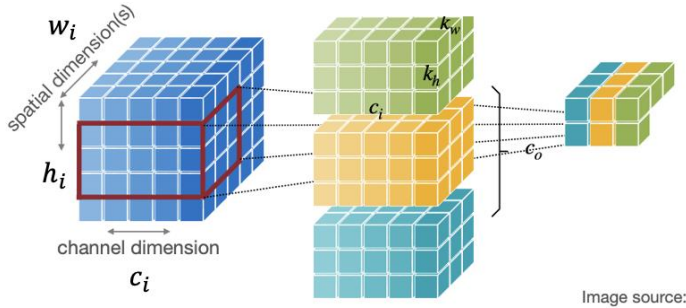


Image source: 1

Notations	
n	Batch Size
c_i	Input Channels
c_o	Output Channels
h_i, h_o	Input/Output Height
w_i, w_o	Input/Output Width
k_h	Kernel Height
k_w	Kernel Width
g	Groups

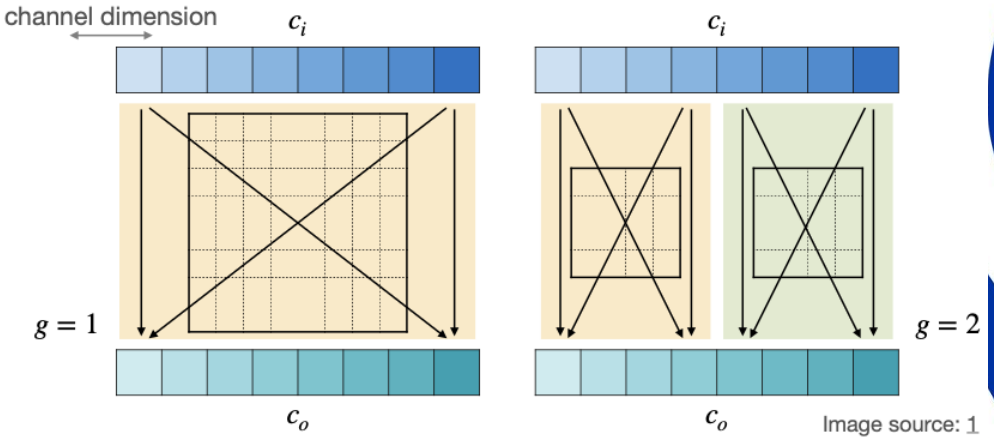


Image source: 1

Depthwise Convolution Layer (Optional)

Independent filter for each channel: $g = c_i = c_o$ **in grouped convolution**

- **Shape of Tensors:**
  - Input Features $\mathbf{X}$: (n, c_i, h_i, w_i)
  - Output Features $\mathbf{Y}$: (n, c_o, h_o, w_o)
  - Weights $\mathbf{W}$: ~~(c_o, c_i, k_h, k_w)~~ (c, k_h, k_w)
  - Bias $\mathbf{b}$: $(c_o,)$

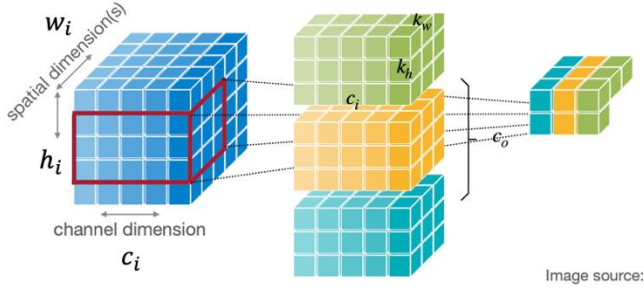


Image source: 1

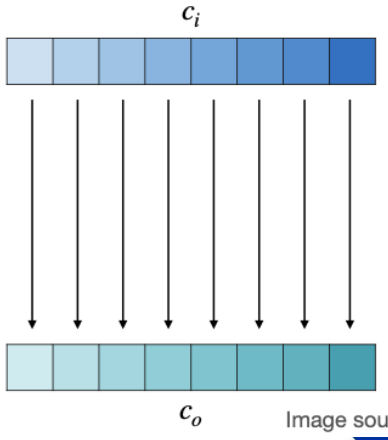


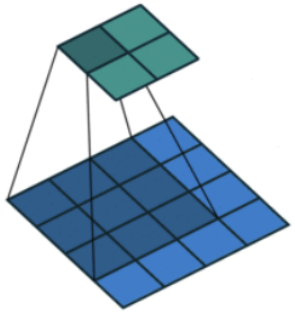
Image source: 1, 2

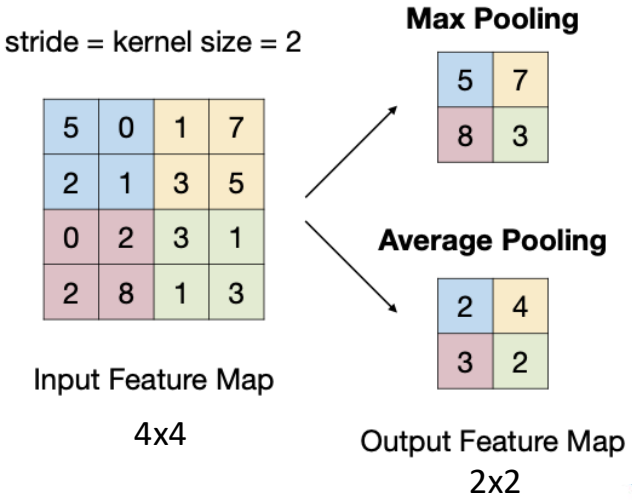
Notations	
n	Batch Size
c_i	Input Channels
c_o	Output Channels
h_i, h_o	Input/Output Height
w_i, w_o	Input/Output Width
k_h	Kernel Height
k_w	Kernel Width
g	Groups

Pooling Layer

Downsample the feature map to a smaller size

- The output neuron pools the features in the receptive field, similar to convolution.
  - Usually, the stride is the same as the kernel size: $s = k$
- Pooling operates over each channel independently.
  - No learnable parameters





Normalization Layer (Optional)

Normalizing the features makes optimization faster.

- Normalization layer normalizes the features as follows,

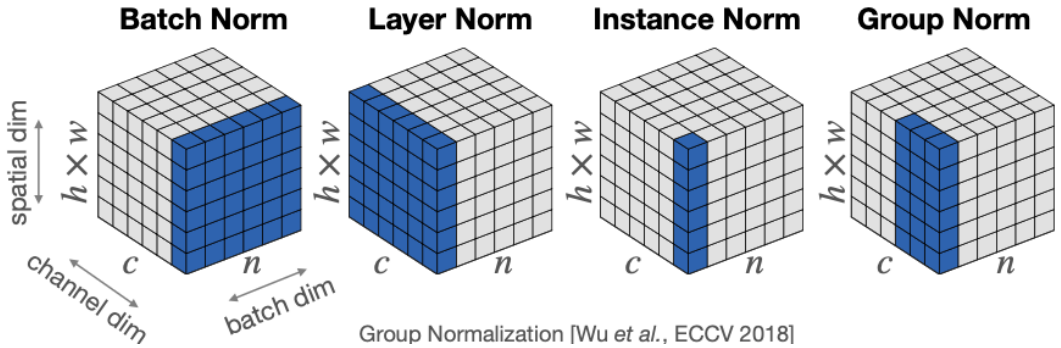
$$\hat{x}_i = \frac{1}{\sigma}\left(x_i - \mu_i\right)$$

$$\mu_i = \frac{1}{m}\sum_{k \in \mathcal{S}_i} x_k$$

$$\sigma_i = \sqrt{\frac{1}{m}\sum_{k \in \mathcal{S}_i}\left(x_k - \mu_i\right)^2 + \epsilon}$$

  - μ_i is the mean, and σ_i is the standard deviation (std) over the set of pixels $\mathcal{S}_i$.
- Then learns a *per-channel* linear transform (trainable scale γ and shift β) to compensate for the possible lost of representational ability.

$$y = \gamma_{i_c}\hat{x}_i + \beta_{i_c}$$

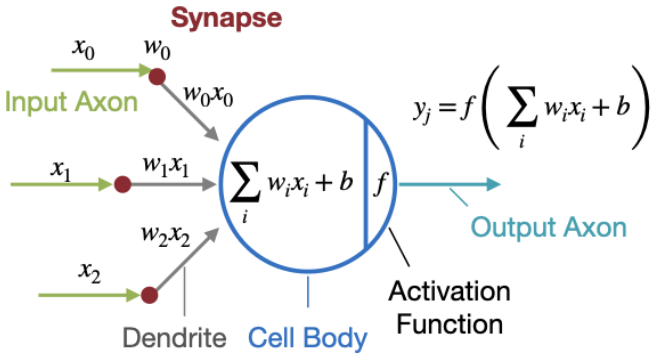


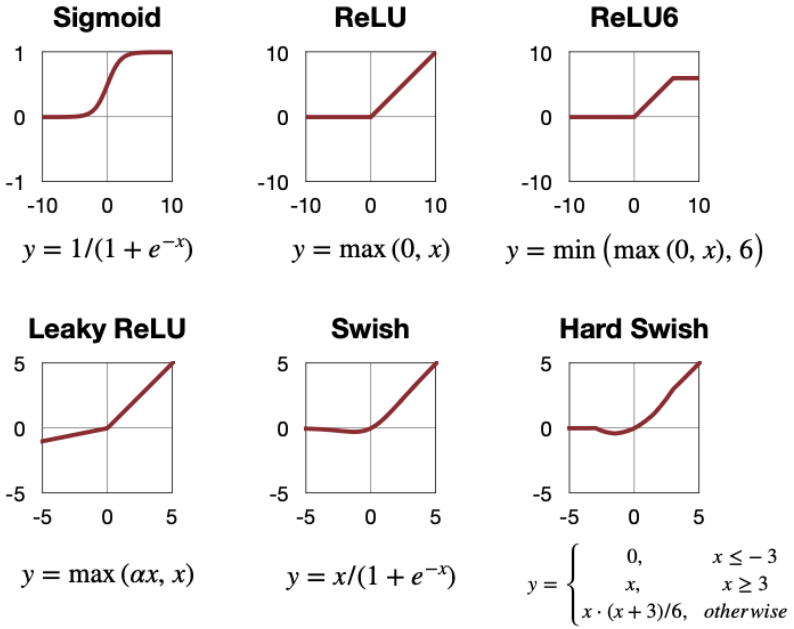
Group Normalization [Wu *et al.*, ECCV 2018]

Different normalizations use different definitions of the set $\mathcal{S}_i$ (colored in blue)

Activation Function

Activation functions are typically non-linear functions

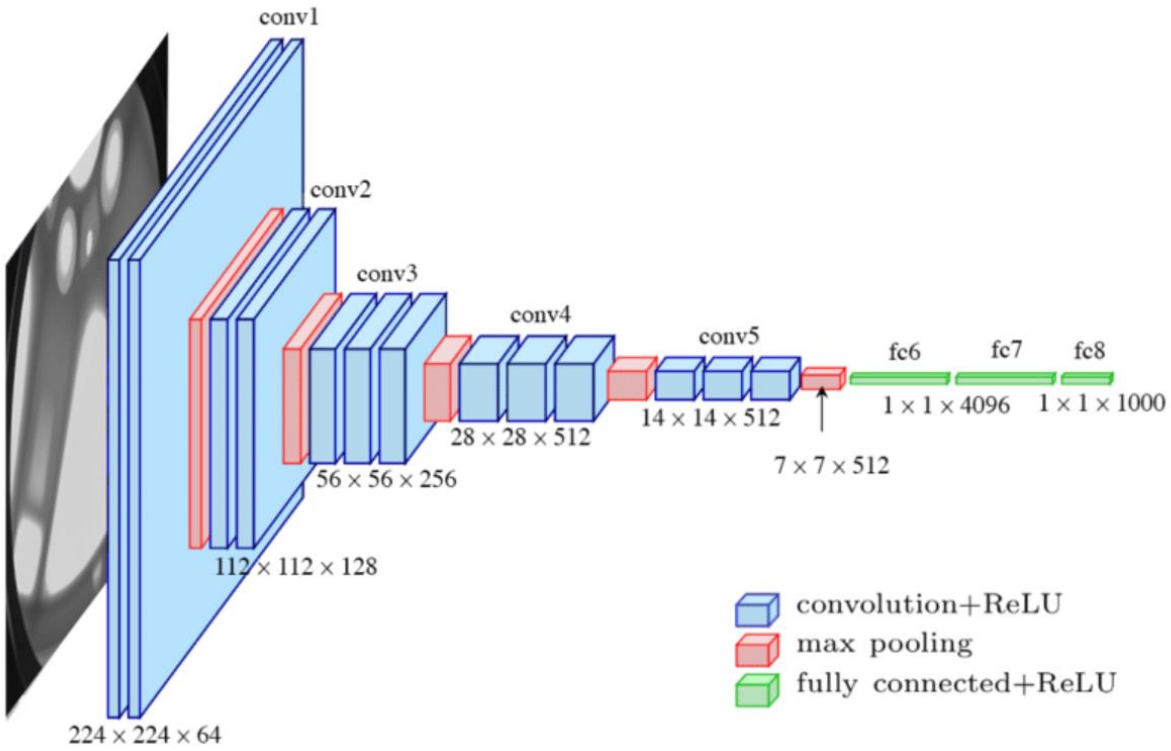




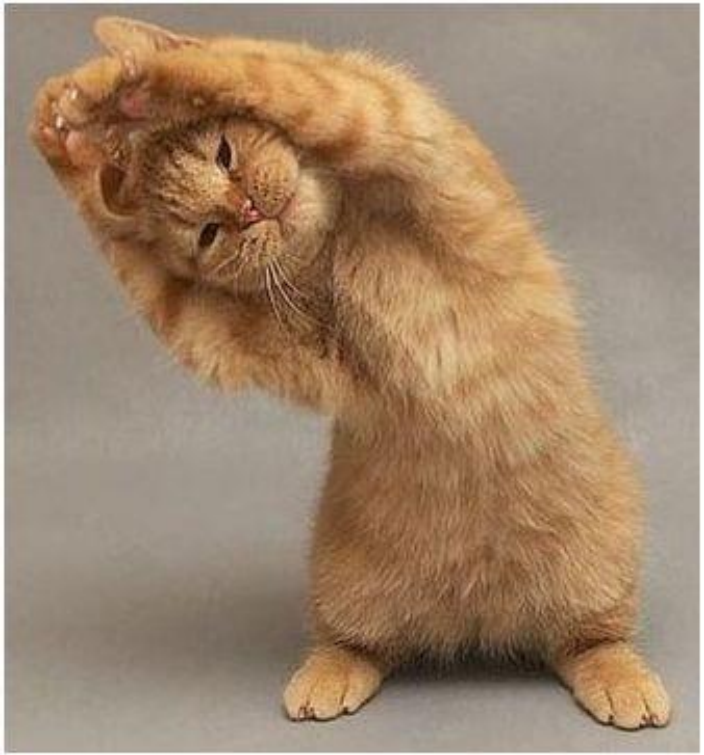
Other Activation Functions: Tanh, GELU, ELU, Mish…

Most common activation functions have NO learnable parameters

CNN Architecture Example



BREAK

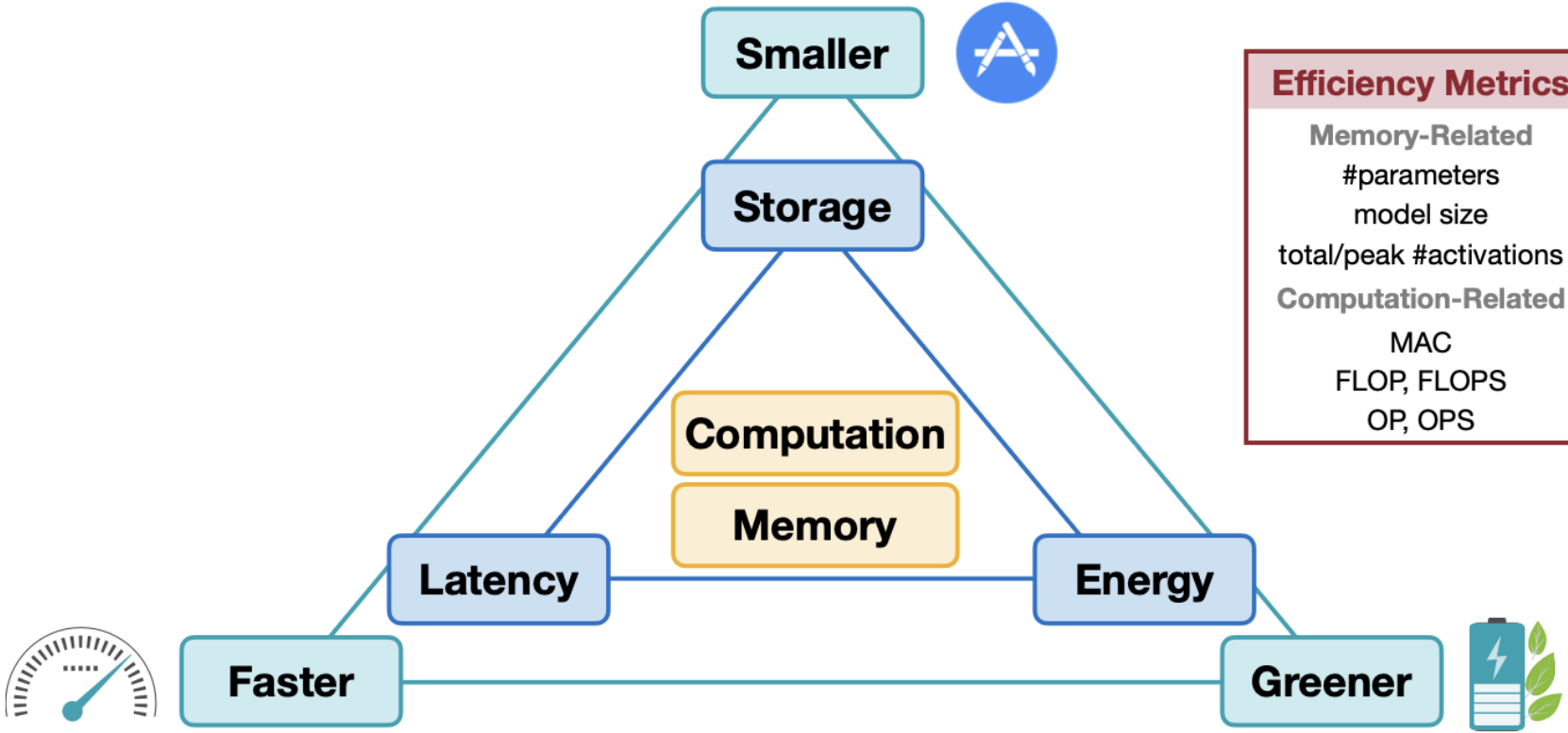


Efficiency Metrics

How should we measure the efficiency of neural networks?



Efficiency of Neural Networks



Efficiency Metrics

Memory-Related

#parameters

model size

total/peak #activations

Computation-Related

MAC

FLOP, FLOPS

OP, OPS

Latency

Measures the delay for a specific task



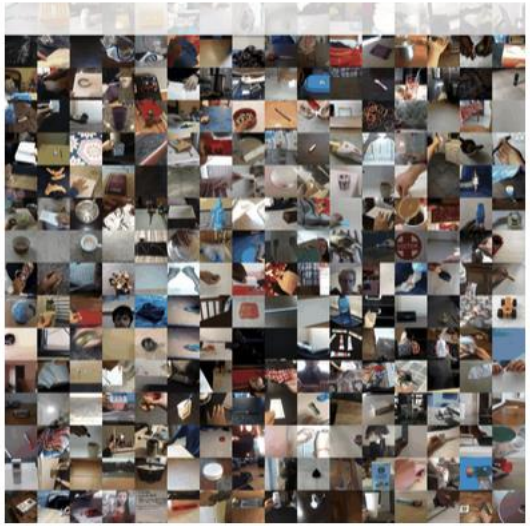
SegFormer
638ms, 82.4mIoU



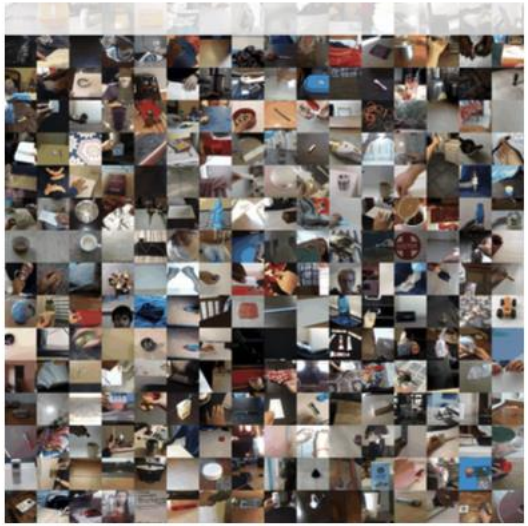
EfficientViT
46ms, 82.7mIoU

Throughput

Measures the rate at which data is processed



Low Throughput = 6.1 video/s



High Throughput = 77.4 video/s

Latency vs. Throughput

- Does higher throughput translate to lower latency? Why?
- Does lower latency translate to higher throughput? Why?



Design 1

Latency: 50 ms
Throughput: 20 image/s



Design 2

Latency: 100 ms
Throughput: 40 image/s

Latency

$$Latency \approx \max\left(T_{computation}, T_{memory}\right)$$

$$T_{\text{computation}} \approx \frac{\text{Number of Operations in Neural Network Model}}{\text{Number of Operations that Processor can Process Per Second}}$$

NN Specification

Hardware Specification

$$T_{\text{memory}} \approx T_{\text{data movement of activations}} + T_{\text{data movement of weights}}$$

$$T_{\text{data movement of weights}} \approx \frac{\text{Neural Network Model Size}}{\text{Memory Bandwidth of Processor}}$$

NN Specification

Hardware Specification

$$T_{\text{data movement of activations}} \approx \frac{\text{Input Activation Size + Output Activation Size}}{\text{Memory Bandwidth of Processor}}$$

NN Specification

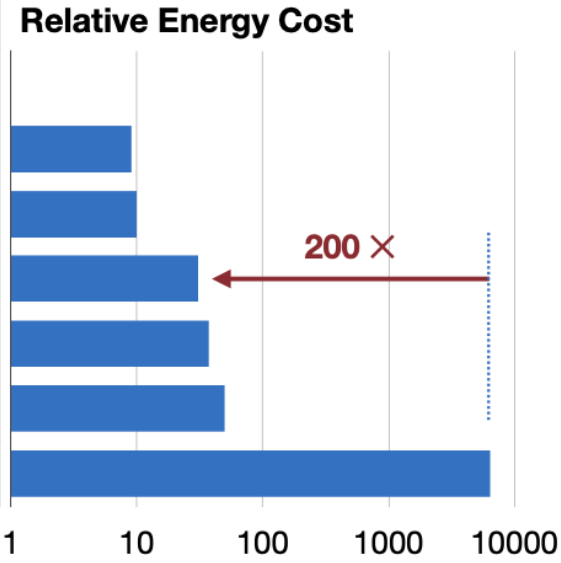
Hardware Specification

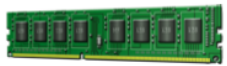
Energy Consumption

Data movement → more memory reference → more energy

Operation	Energy [pJ]
32 bit int ADD	0.1
32 bit float ADD	0.9
32 bit Register File	1
32 bit int MULT	3.1
32 bit float MULT	3.7
32 bit SRAM Cache	5
32 bit DRAM Memory	640

Rough Energy Cost For Various Operations in 45nm 0.9V

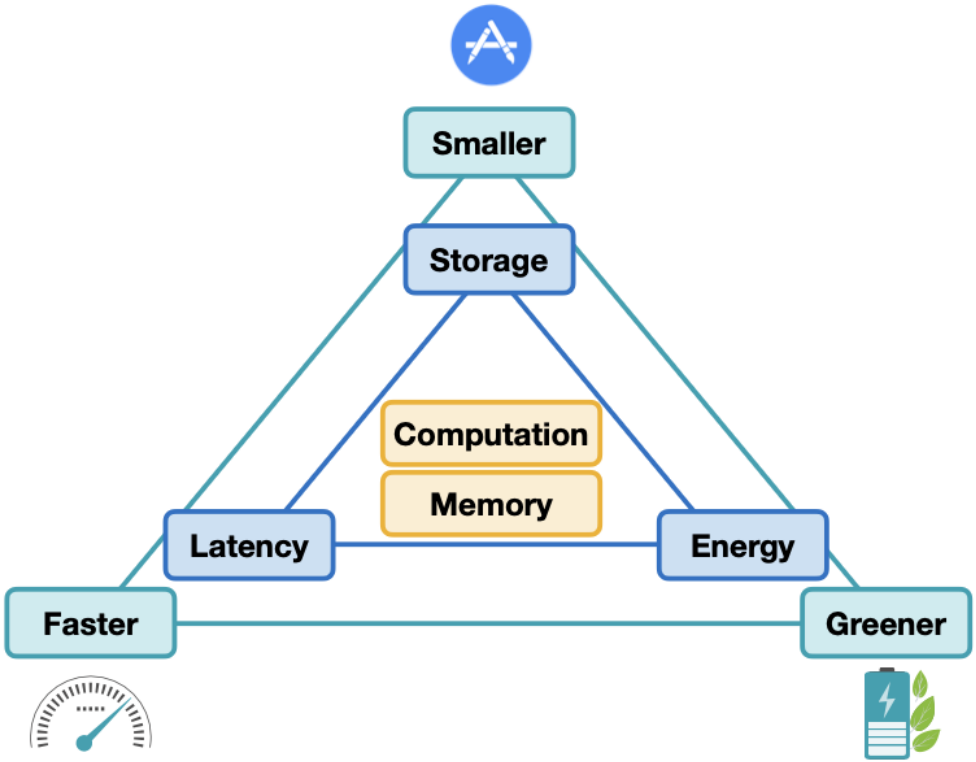


1  = 200 ×+

This image is in the public domain

Computing's Energy Problem (and What We Can Do About it) [Horowitz, M., IEEE ISSCC 2014]

Efficiency of Neural Networks



Efficiency Metrics

Memory-Related

#parameters

model size

total/peak #activations

Computation-Related

MAC

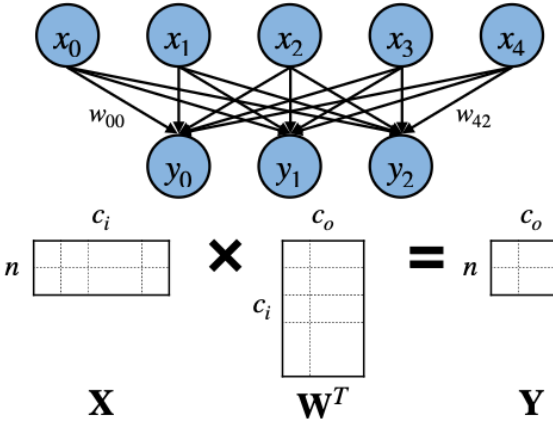
FLOP, FLOPS

OP, OPS

Number of Parameters (# Parameters)

#Parameters is the parameter (synapse/weight) count of the given neural network, *i.e.*, the number of elements in the weight tensors.

Layer	#Parameters (bias is ignored)
Linear Layer	$c_o \cdot c_i$
Convolution	
Grouped Convolution	
Depthwise Convolution	



Notations	
n	Batch Size
c_i	Input Channels
c_o	Output Channels
h_i, h_o	Input/Output Height
w_i, w_o	Input/Output Width
k_h, k_w	Kernel Height/Width
g	Groups

Number of Parameters (# Parameters)

#Parameters is the parameter (synapse/weight) count of the given neural network, *i.e.*, the number of elements in the weight tensors.

Layer	#Parameters (bias is ignored)
Linear Layer	$c_o \cdot c_i$
Convolution	$c_o \cdot c_i \cdot k_h \cdot k_w$
Grouped Convolution	
Depthwise Convolution	

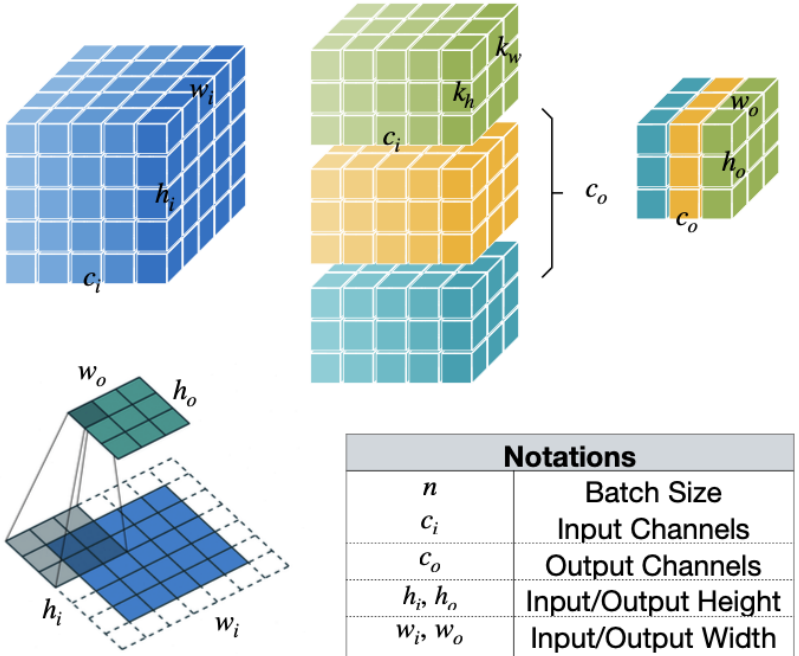


Image source: 1

Notations	
n	Batch Size
c_i	Input Channels
c_o	Output Channels
h_i, h_o	Input/Output Height
w_i, w_o	Input/Output Width
k_h, k_w	Kernel Height/Width
g	Groups

Number of Parameters (Optional)

#Parameters is the parameter (synapse/weight) count of the given neural network, *i.e.*, the number of elements in the weight tensors.

Layer	#Parameters (bias is ignored)
Linear Layer	$c_o \cdot c_i$
Convolution	$c_o \cdot c_i \cdot k_h \cdot k_w$
Grouped Convolution	$c_o/g \cdot c_i/g \cdot k_h \cdot k_w \cdot g = c_o \cdot c_i \cdot k_h \cdot k_w/g$
Depthwise Convolution	$c_o \cdot k_h \cdot k_w$

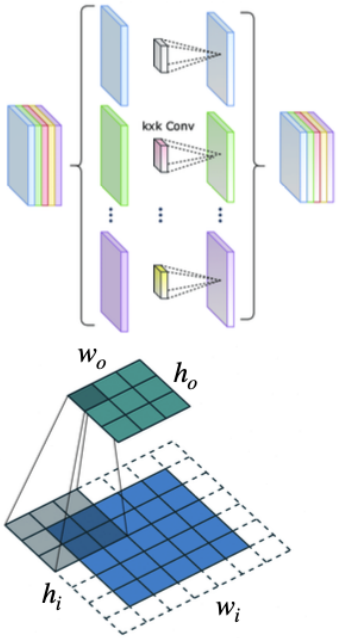
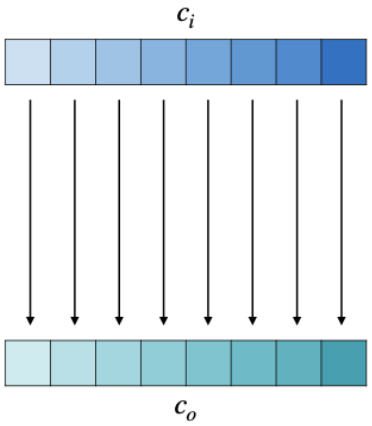
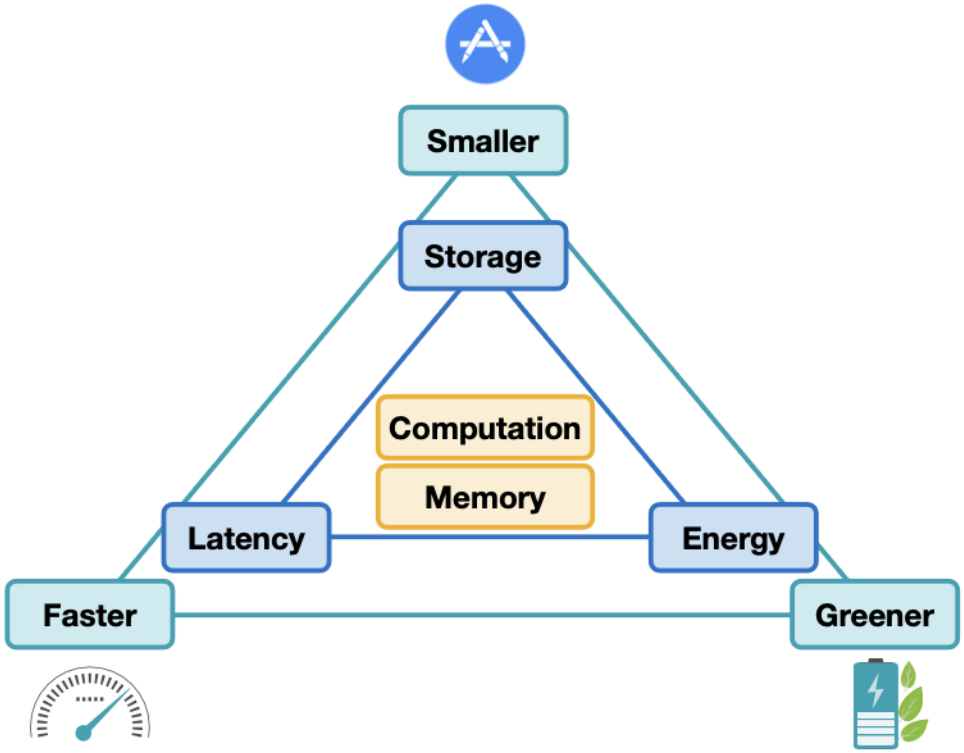


Image source: 1



Notations	
n	Batch Size
c_i	Input Channels
c_o	Output Channels
h_i, h_o	Input/Output Height
w_i, w_o	Input/Output Width
k_h, k_w	Kernel Height/Width
g	Groups

Efficiency of Neural Networks



Efficiency Metrics

Memory-Related

#parameters

model size

total/peak #activations

Computation-Related

MAC

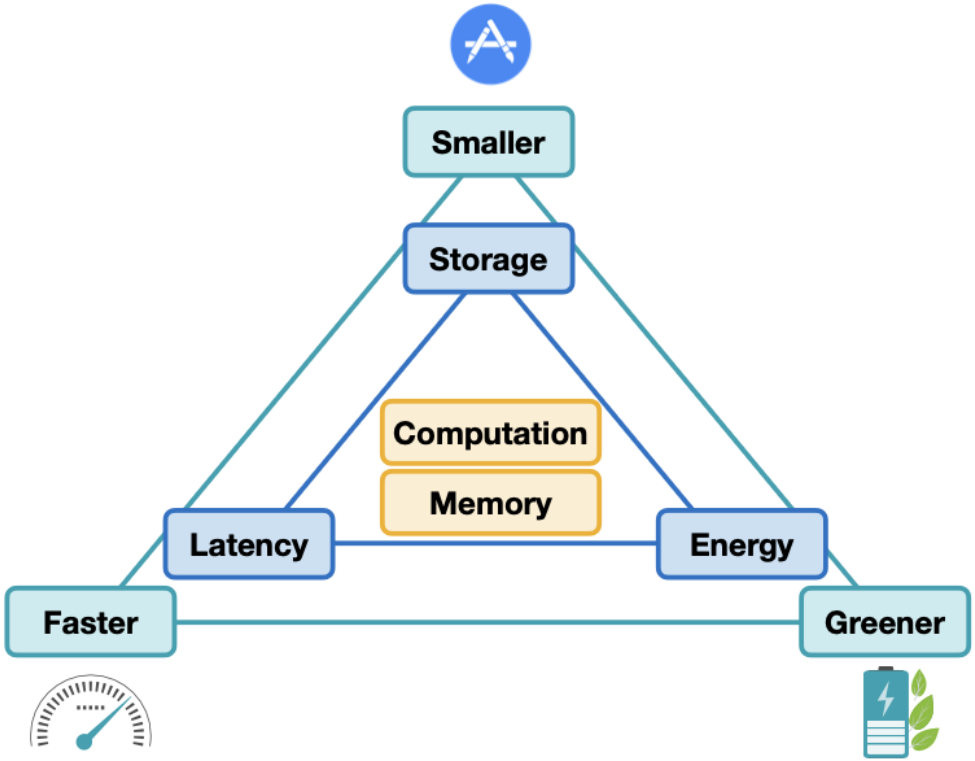
FLOP, FLOPS

OP, OPS

Model Size

- Model size measures the storage for the weights of the given neural network.
  - The common units for model size are: MB (megabyte), KB (kilobyte), bits.
- In general, if the whole neural network uses the same data type (*e.g.*, floating-point),
  - $Model\ \ Size = \#Parameters \cdot Bit\ Width$
  - Example: AlexNet has 61M parameters.
    - If all weights are stored with 32-bit numbers, total storage will be about
      - 61M × 4 Bytes (32 bits) = 244 MB (244×10^6 Bytes)
    - If all weights are stored with 8-bit numbers, total storage will be about
      - 61M × 1 Byte (8 bits) = 61 MB

Efficiency of Neural Networks



Efficiency Metrics

Memory-Related

#parameters

model size

total/peak #activations

Computation-Related

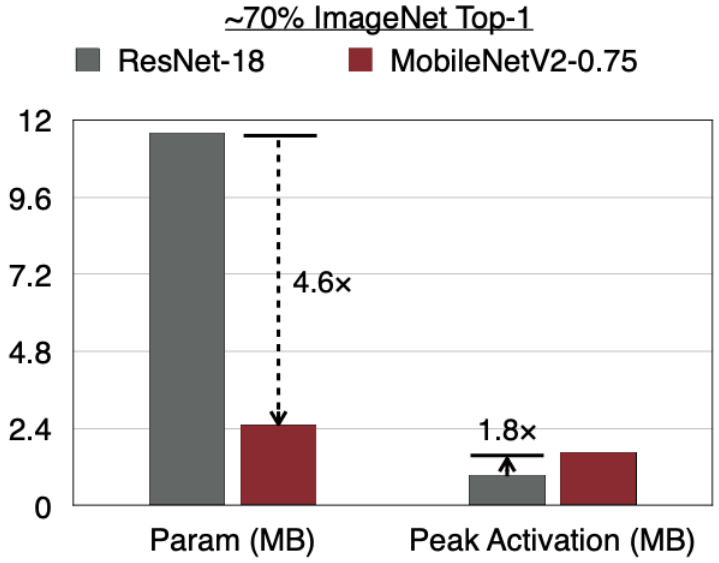
MAC

FLOP, FLOPS

OP, OPS

Number of Activations (#Activations)

#Activations is the memory bottleneck in CNN inference, not #parameters

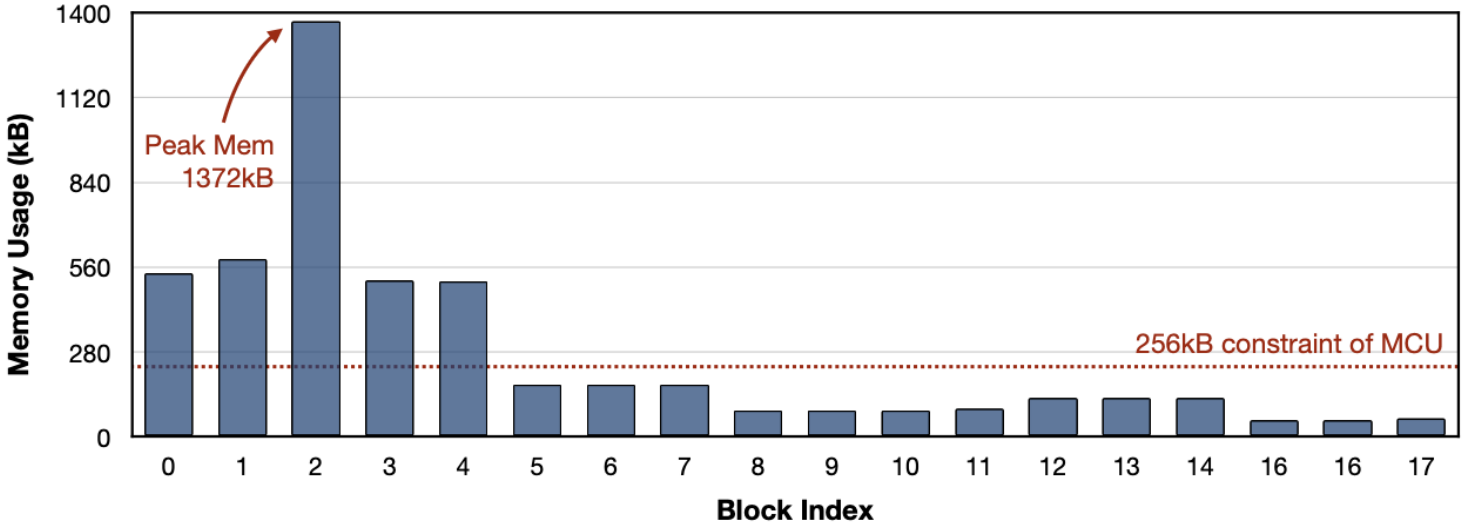


*** All parameters and activations are Integer numbers (8 bits).**

MCUNet: Tiny Deep Learning on IoT Devices [Lin *et al.*, NeurIPS 2020]

Number of Activations (#Activations)

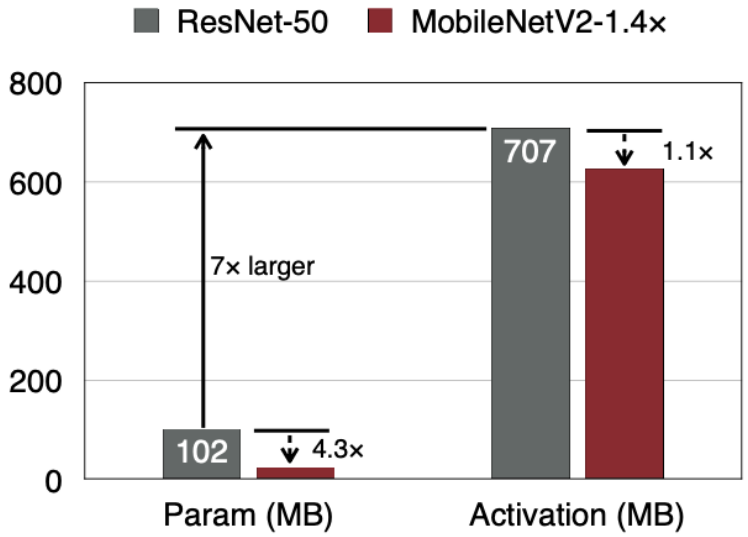
Imbalanced memory distribution of MobileNetV2



MCUNet: Tiny Deep Learning on IoT Devices [Lin *et al.*, NeurIPS 2020]

Number of Activations (#Activations)

#Activations is the memory bottleneck in training, not #parameters

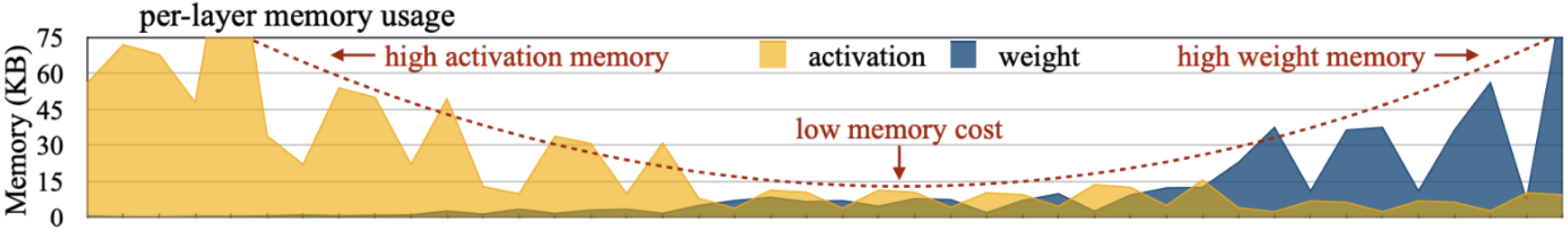


The main memory bottleneck does not improve much.

*** All parameters and activations are Floating-Point numbers (32 bits).**

Number of Activations (#Activations)

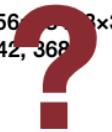
Activations and weight memory distribution of CNN



MCUNet: Tiny Deep Learning on IoT Devices [Lin *et al.*, NeurIPS 2020]

AlexNet: #Parameters

AlexNet	C × H × W	#Parameters (bias is ignored)
Image (3×224×224)	3×224×224	
11×11 Conv, channel 96, stride 4, pad 2	96×55×55	96×3×11×11 = 24, 848
3×3 MaxPool, stride 2	96×27×27	
5×5 Conv, channel 256, pad 2, groups 2	256×27×27	256×96×5×5 / 2 = 307, 200
3×3 MaxPool, stride 2	256×13×13	
3×3 Conv, channel 384, pad 1	384×13×13	384×256×3×3 = 884, 736
3×3 Conv, channel 384, pad 1, groups 2	384×13×13	384×384×3×3 / 2 = 663, 552
3×3 Conv, channel 256, pad 1, groups 2	256×13×13	256×[illegible]×3×3 / 2 = 442, 368
3×3 MaxPool, stride 2	256×6×6	
Linear, channel 4096	4096	4096×(256×6×6) = 37, 748, 736
Linear, channel 4096	4096	4096×4096 = 16, 777, 216
Linear, channel 1000	1000	1000×4096 = 4, 096, 000



Layer	#Parameters
Linear Layer	$c_o \cdot c_i$
Convolution	$c_o \cdot c_i \cdot k_h \cdot k_w$
Grouped Convolution	$c_o \cdot c_i \cdot k_h \cdot k_w / g$
Depthwise Convolution	$c_o \cdot k_h \cdot k_w$

61M in total

ImageNet Classification with Deep Convolutional Neural Networks [Krizhevsky *et al.*, NeurIPS 2012]

AlexNet: #Activations

AlexNet	C × H × W	
Image (3×224×224)	3×224×224	=150,528
11×11 Conv, channel 96, stride 4, pad 2	96×55×55	=290,400
3×3 MaxPool, stride 2	96×27×27	=69,984
5×5 Conv, channel 256, pad 2, groups 2	256×27×27	=186,624
3×3 MaxPool, stride 2	256×13×13	=43,264
3×3 Conv, channel 384, pad 1	384×13×13	=64,896
3×3 Conv, channel 384, pad 1, groups 2	384×13×13	=64,896
3×3 Conv, channel 256, pad 1, groups 2	256×13×13	=43,264
3×3 MaxPool, stride 2	256×6×6	=9,216
Linear, channel 4096	4096	=4,096
Linear, channel 4096	4096	=4,096
Linear, channel 1000	1000	=1,000

Total #Activation: 932,264

Peak #Activation:

≈ #input activation + #output activation

= 150,528 + 290,400 = 440,928

ImageNet Classification with Deep Convolutional Neural Networks [Krizhevsky *et al.*, NeurIPS 2012]

Quiz 2

Quiz#, Name, email.

